

# **WEB TECHNOLOGY LAB MANUAL**

*Form Validation, Bootstrap 5 & AJAX/jQuery Practical Exercises( 6 to 10)*

## **Table of Contents**

Experiment 6: JavaScript Exercises for Form Validation

Experiment 7: Student Registration Form Using JavaScript

Experiment 8: RSVP Form Design Using Bootstrap Form Controls

Experiment 9: Responsive Image Grid Using Bootstrap 5

Experiment 10: AJAX-based Dynamic Content Loading with jQuery Fading Effects

# Experiment 6: JavaScript Exercises for Form Validation

## Aim

To validate HTML form fields on the client side using JavaScript, checking for required input, correct email format, minimum password length, matching passwords, and valid phone numbers.

## Theory / Description

Client-side form validation uses JavaScript to check user input before it is submitted, providing immediate feedback and reducing unnecessary server requests.

Common validation techniques include checking string length, using regular expressions (RegExp) to verify patterns such as email addresses and phone numbers, and comparing two fields (e.g., password and confirm password) for equality.

The onsubmit event handler of a <form> can call a JavaScript function that returns true (allow submission) or false (block submission), while error messages are displayed dynamically by updating the innerText of dedicated <div> elements.

## Procedure (Step-by-Step)

1. Create a form with fields for Name, Email, Password, Confirm Password, and Phone Number, each followed by an empty <div> to hold error messages.
2. Set the form's onsubmit attribute to call a validateForm() function and return its result.
3. Inside validateForm(), retrieve each field's value using document.getElementById().value.
4. Validate the name (minimum length, letters only), email (regex pattern), password (minimum 6 characters), confirm password (must match), and phone number (exactly 10 digits).
5. For each failed check, display an error message in the corresponding <div> and add an 'invalid' CSS class to highlight the field in red.
6. If all checks pass, display a success message; otherwise show a general 'fix the errors' message.
7. Test the form with both valid and invalid input to confirm the validation logic works correctly.

## Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Lab 6 - JavaScript Form Validation</title>
<style>
  body {
    font-family: Arial, sans-serif;
    background: #eef2f5;
    display: flex;
    justify-content: center;
    padding: 30px;
    margin: 0;
  }
  form {
    background: white;
    width: 480px;
    padding: 25px 30px;
    border-radius: 8px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
  }
  h2 {
    text-align: center;
    color: #2c3e50;
    margin-top: 0;
  }
  label {
    display: block;
    margin-top: 14px;
    margin-bottom: 4px;
    font-weight: bold;
    color: #34495e;
  }
```

```

}
input {
  width: 100%;
  padding: 8px 10px;
  border: 1px solid #ccc;
  border-radius: 5px;
  font-size: 14px;
}
input.invalid {
  border-color: #e74c3c;
  background: #fdecea;
}
.error {
  color: #e74c3c;
  font-size: 12px;
  margin-top: 3px;
  min-height: 14px;
}
}
button {
  margin-top: 20px;
  width: 100%;
  background: #2980b9;
  color: white;
  padding: 10px;
  border: none;
  border-radius: 5px;
  font-size: 16px;
  cursor: pointer;
}
#statusMsg {
  text-align: center;
  margin-top: 12px;
  font-weight: bold;
}
}
</style>
</head>
<body>
  <form id="validationForm" onsubmit="return validateForm()" novalidate>
    <h2>Form Validation Exercise</h2>

    <label for="name">Full Name</label>
    <input type="text" id="name" value="R">
    <div class="error" id="nameError"></div>

    <label for="email">Email Address</label>
    <input type="text" id="email" value="rohan@com">
    <div class="error" id="emailError"></div>

    <label for="password">Password</label>
    <input type="password" id="password" value="123">
    <div class="error" id="passwordError"></div>

    <label for="confirmPassword">Confirm Password</label>
    <input type="password" id="confirmPassword" value="1234">
    <div class="error" id="confirmError"></div>

    <label for="phone">Phone Number</label>
    <input type="text" id="phone" value="98765abc10">
    <div class="error" id="phoneError"></div>

    <button type="submit">Submit</button>
    <div id="statusMsg"></div>
  </form>

  <script>
    function showError(fieldId, errorId, message) {
      document.getElementById(errorId).innerText = message;
      document.getElementById(fieldId).classList.toggle('invalid', message !== "");
    }
  </script>

```

```

function validateForm() {
    let isValid = true;

    // Name validation: at least 3 characters, letters/spaces only
    const name = document.getElementById('name').value.trim();
    if (name.length < 3 || !/^[A-Za-z ]+$/.test(name)) {
        showError('name', 'nameError', 'Name must be at least 3 letters (no numbers/symbols).');
        isValid = false;
    } else {
        showError('name', 'nameError', '');
    }

    // Email validation using regex
    const email = document.getElementById('email').value.trim();
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailPattern.test(email)) {
        showError('email', 'emailError', 'Enter a valid email address (e.g. name@domain.com).');
        isValid = false;
    } else {
        showError('email', 'emailError', '');
    }

    // Password validation: min 6 characters
    const password = document.getElementById('password').value;
    if (password.length < 6) {
        showError('password', 'passwordError', 'Password must be at least 6 characters long. ');
        isValid = false;
    } else {
        showError('password', 'passwordError', '');
    }

    // Confirm password must match
    const confirmPassword = document.getElementById('confirmPassword').value;
    if (confirmPassword !== password) {
        showError('confirmPassword', 'confirmError', 'Passwords do not match. ');
        isValid = false;
    } else {
        showError('confirmPassword', 'confirmError', '');
    }

    // Phone validation: exactly 10 digits
    const phone = document.getElementById('phone').value.trim();
    if (!/^\d{10}$/.test(phone)) {
        showError('phone', 'phoneError', 'Phone number must contain exactly 10 digits. ');
        isValid = false;
    } else {
        showError('phone', 'phoneError', '');
    }

    const status = document.getElementById('statusMsg');
    if (isValid) {
        status.style.color = '#27ae60';
        status.innerText = 'Form submitted successfully!';
    } else {
        status.style.color = '#e74c3c';
        status.innerText = 'Please fix the highlighted errors.';
    }
    return false; // prevent actual submission for demo
}

// Run validation once on load so the output screenshot shows the result
window.onload = validateForm;
</script>
</body>
</html>

```

## Output

## Form Validation Exercise

**Full Name**

R

Name must be at least 3 letters (no numbers/symbols).

**Email Address**

rohan@com

Enter a valid email address (e.g. name@domain.com).

**Password**

...

Password must be at least 6 characters long.

**Confirm Password**

....

Passwords do not match.

**Phone Number**

98765abc10

Phone number must contain exactly 10 digits.

Submit

Please fix the highlighted errors.

Fig. 6: Validation errors displayed for invalid sample input

# Experiment 7: Student Registration Form Using JavaScript

## Aim

To design a student registration form that uses JavaScript to calculate age from date of birth, dynamically populate a branch dropdown based on the selected course, and display a registration summary using DOM manipulation.

## Theory / Description

JavaScript can be used to build interactive, data-driven forms. Dynamic dropdowns are implemented by listening for a change event on one <select> element and repopulating another <select> element's <option> list based on the chosen value, using a lookup object (e.g., an object mapping courses to arrays of branches).

Date calculations, such as computing age from a date of birth, are performed using JavaScript's built-in Date object, comparing the year, month, and day components of the current date and the birth date.

On form submission, JavaScript can read all field values and dynamically insert them into a hidden summary section by setting the innerText of placeholder elements, then revealing that section by changing its CSS display property.

## Procedure (Step-by-Step)

8. Create the registration form with fields for Name, Date of Birth, Course (dropdown), Branch (dropdown), and Email.
9. Define a JavaScript object (branchMap) that maps each course to an array of valid branches/specializations.
10. Write a populateBranches() function that clears and refills the Branch dropdown whenever the Course dropdown changes.
11. Write a calculateAge() function that computes the student's age from the selected date of birth using the Date object.
12. Write a registerStudent() function, triggered on form submit, that reads all field values and writes them into a hidden 'Registration Summary' card, then makes the card visible.
13. Test the form by changing the course (to confirm the branch list updates) and submitting to verify the summary card displays correctly.

## Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Lab 7 - Student Registration Form</title>
<style>
  body {
    font-family: Arial, sans-serif;
    background: #eef2f5;
    display: flex;
    justify-content: center;
    gap: 25px;
    padding: 30px;
    margin: 0;
    flex-wrap: wrap;
  }
  form {
    background: white;
    width: 420px;
    padding: 25px 30px;
    border-radius: 8px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
  }
  h2 { text-align: center; color: #2c3e50; margin-top: 0; }
  label {
    display: block;
    margin-top: 14px;
    margin-bottom: 6px;
    font-weight: bold;
    color: #34495e;
  }
  input, select {
    width: 100%;
```

```

padding: 8px 10px;
border: 1px solid #ccc;
border-radius: 5px;
font-size: 14px;
}
button {
margin-top: 20px;
width: 100%;
background: #27ae60;
color: white;
padding: 10px;
border: none;
border-radius: 5px;
font-size: 16px;
cursor: pointer;
}
#ageDisplay {
font-size: 12px;
color: #2980b9;
margin-top: 4px;
}
#summaryCard {
background: white;
width: 320px;
padding: 20px 25px;
border-radius: 8px;
box-shadow: 0 4px 12px rgba(0,0,0,0.1);
border-top: 5px solid #27ae60;
}
#summaryCard h3 { color: #27ae60; margin-top: 0; }
#summaryCard table td { padding: 5px 0; vertical-align: top; }
#summaryCard table td.label { font-weight: bold; color: #34495e; width: 90px; }
</style>
</head>
<body>
<form id="regForm" onsubmit="return registerStudent()">
  <h2>Student Registration Form</h2>

  <label for="sname">Student Name</label>
  <input type="text" id="sname" value="Ananya Verma">

  <label for="dob">Date of Birth</label>
  <input type="date" id="dob" value="2006-08-15" onchange="calculateAge()">
  <div id="ageDisplay"></div>

  <label for="course">Course</label>
  <select id="course" onchange="populateBranches()">
    <option value="BE">B.E.</option>
    <option value="BCA">BCA</option>
    <option value="MBA">MBA</option>
  </select>

  <label for="branch">Branch / Specialization</label>
  <select id="branch"></select>

  <label for="semail">Email Address</label>
  <input type="email" id="semail" value="ananya.verma@example.com">

  <button type="submit">Register</button>
</form>

<div id="summaryCard" style="display:none;">
  <h3>Registration Summary</h3>
  <table>
    <tr><td class="label">Name</td><td id="sumName"></td></tr>
    <tr><td class="label">Age</td><td id="sumAge"></td></tr>
    <tr><td class="label">Course</td><td id="sumCourse"></td></tr>
    <tr><td class="label">Branch</td><td id="sumBranch"></td></tr>
    <tr><td class="label">Email</td><td id="sumEmail"></td></tr>
  </table>

```



```

</div>

<script>
  const branchMap = {
    BE: ["Computer Science", "Electronics", "Mechanical", "Civil"],
    BCA: ["General", "Data Science"],
    MBA: ["Finance", "Marketing", "HR"]
  };

  function populateBranches() {
    const course = document.getElementById('course').value;
    const branchSelect = document.getElementById('branch');
    branchSelect.innerHTML = "";
    branchMap[course].forEach(b => {
      const opt = document.createElement('option');
      opt.value = b;
      opt.text = b;
      branchSelect.appendChild(opt);
    });
  }

  function calculateAge() {
    const dob = new Date(document.getElementById('dob').value);
    const today = new Date();
    let age = today.getFullYear() - dob.getFullYear();
    const m = today.getMonth() - dob.getMonth();
    if (m < 0 || (m === 0 && today.getDate() < dob.getDate())) age--;
    document.getElementById('ageDisplay').innerText = "Calculated Age: " + age + " years";
    return age;
  }

  function registerStudent() {
    const name = document.getElementById('sname').value;
    const age = calculateAge();
    const course = document.getElementById('course').value;
    const branch = document.getElementById('branch').value;
    const email = document.getElementById('semail').value;

    document.getElementById('sumName').innerText = name;
    document.getElementById('sumAge').innerText = age + " years";
    document.getElementById('sumCourse').innerText = course;
    document.getElementById('sumBranch').innerText = branch;
    document.getElementById('sumEmail').innerText = email;
    document.getElementById('summaryCard').style.display = "block";
    return false; // prevent actual page reload for demo
  }

  // Initialize on load so the screenshot shows a completed demo
  window.onload = function() {
    populateBranches();
    calculateAge();
    registerStudent();
  };
</script>
</body>
</html>

```

## Output

### Student Registration Form

**Student Name**

**Date of Birth**

Calculated Age: 19 years

**Course**

**Branch / Specialization**

**Email Address**

Register

#### Registration Summary

|               |                          |
|---------------|--------------------------|
| <b>Name</b>   | Ananya Verma             |
| <b>Age</b>    | 19 years                 |
| <b>Course</b> | BE                       |
| <b>Branch</b> | Computer Science         |
| <b>Email</b>  | ananya.verma@example.com |

*Fig. 7: Student registration form with the generated summary card*

# Experiment 8: RSVP Form Design Using Bootstrap Form Controls

## Aim

To design an RSVP (event response) form using Bootstrap 5's pre-built form control classes for a clean, responsive, and consistent layout without writing custom CSS from scratch.

## Theory / Description

Bootstrap is a popular CSS framework that provides ready-made classes for layout (container, row, col-\*), components (card), and form controls (form-control, form-select, form-check) that are responsive and consistent across browsers by default.

Bootstrap form classes such as form-control (for text/email inputs and textareas), form-select (for dropdowns), form-check / form-check-input (for radio buttons and checkboxes), and utility classes like btn, btn-primary, and d-grid make it possible to build professional-looking forms quickly.

This exercise wraps the RSVP form inside a Bootstrap card component, which provides a bordered, shadowed container with card-header and card-body sections.

## Procedure (Step-by-Step)

14. Include the Bootstrap CSS file (bootstrap.min.css) in the <head> of the HTML document.
15. Create a card container using the card, card-header, and card-body classes to hold the RSVP form.
16. Add a Name field and Email field using <input class="form-control">, each wrapped in a mb-3 div with a form-label.
17. Add an 'Attending?' choice using two radio buttons styled with form-check and form-check-input classes.
18. Add a 'Number of Guests' dropdown using <select class="form-select">.
19. Add Meal Preference options using inline checkboxes (form-check form-check-inline).
20. Add a message <textarea class="form-control"> and a full-width submit button using the d-grid and btn btn-primary btn-lg classes.
21. Save and preview the form in a browser to confirm Bootstrap's default styling has been applied correctly.

## Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Lab 8 - RSVP Form (Bootstrap)</title>
<link rel="stylesheet" href="vendor/bootstrap.min.css">
<style>
  body { background: #f4f6f8; padding: 40px 0; }
  .rsvp-card {
    max-width: 550px;
    margin: 0 auto;
    box-shadow: 0 4px 15px rgba(0,0,0,0.1);
    border: none;
  }
  .card-header {
    background: linear-gradient(135deg, #6a11cb, #2575fc);
    color: white;
    text-align: center;
    padding: 25px;
  }
</style>
</head>
<body>
  <div class="card rsvp-card">
    <div class="card-header">
      <h2 class="mb-0">You're Invited!</h2>
      <p class="mb-0">Please RSVP for the Annual Alumni Meet</p>
    </div>
    <div class="card-body p-4">
      <form>
        <div class="mb-3">
```

```

    <label class="form-label">Full Name</label>
    <input type="text" class="form-control" placeholder="Enter your name">
  </div>

  <div class="mb-3">
    <label class="form-label">Email Address</label>
    <input type="email" class="form-control" placeholder="you@example.com">
  </div>

  <div class="mb-3">
    <label class="form-label">Will you attend?</label>
    <div class="form-check">
      <input class="form-check-input" type="radio" name="attend" id="attendYes" checked>
      <label class="form-check-label" for="attendYes">Joyfully Accept</label>
    </div>
    <div class="form-check">
      <input class="form-check-input" type="radio" name="attend" id="attendNo">
      <label class="form-check-label" for="attendNo">Regretfully Decline</label>
    </div>
  </div>

  <div class="mb-3">
    <label class="form-label">Number of Guests</label>
    <select class="form-select">
      <option>Just Me</option>
      <option selected>Me + 1</option>
      <option>Me + 2</option>
      <option>Me + 3</option>
    </select>
  </div>

  <div class="mb-3">
    <label class="form-label">Meal Preference</label>
    <div class="form-check form-check-inline">
      <input class="form-check-input" type="checkbox" id="veg" checked>
      <label class="form-check-label" for="veg">Vegetarian</label>
    </div>
    <div class="form-check form-check-inline">
      <input class="form-check-input" type="checkbox" id="nonveg">
      <label class="form-check-label" for="nonveg">Non-Vegetarian</label>
    </div>
    <div class="form-check form-check-inline">
      <input class="form-check-input" type="checkbox" id="vegan">
      <label class="form-check-label" for="vegan">Vegan</label>
    </div>
  </div>

  <div class="mb-3">
    <label class="form-label">Message for the Host</label>
    <textarea class="form-control" rows="3" placeholder="Write a short message"></textarea>
  </div>

  <div class="d-grid">
    <button type="submit" class="btn btn-primary btn-lg">Send RSVP</button>
  </div>
</form>
</div>
</body>
</html>

```

## Output

**You're Invited!**

Please RSVP for the Annual Alumni Meet

Full Name

Email Address

Will you attend?

☒ Joyfully Accept

☐ Regretfully Decline

Number of Guests

Meal Preference

☒ Vegetarian

☐ Non-Vegetarian

☐ Vegan

Message for the Host

*Fig. 8: RSVP form styled with Bootstrap 5 form controls*

# Experiment 9: Responsive Image Grid Using Bootstrap 5

## Aim

To create a responsive image/photo grid gallery using Bootstrap 5's grid system, which automatically rearranges from a multi-column layout on desktop to a stacked layout on smaller screens.

## Theory / Description

Bootstrap's grid system is based on a 12-column layout using container, row, and col-\* classes. Responsive breakpoint classes such as col-6 and col-md-4 allow an element's width to change automatically depending on the screen size, without writing custom media queries.

In this exercise, col-6 col-md-4 makes each gallery item take up half the row width (2 per row) on extra-small/small screens, and one-third of the row width (3 per row) on medium screens and above.

The g-3 class on the row adds consistent gutter spacing (gaps) between grid items, and each grid cell contains a styled div acting as an image placeholder with a caption overlay.

## Procedure (Step-by-Step)

22. Include the Bootstrap CSS file in the <head> of the HTML document.
23. Create a container div and, inside it, a row with the g-3 gutter class.
24. Add six column divs using the class col-6 col-md-4, each representing one photo tile.
25. Inside each column, add a styled div (grid-item) representing an image, along with a caption overlay showing the photo's title.
26. Apply distinct background gradients to each tile using custom CSS classes for visual variety.
27. Preview the page at a wide (desktop) browser width to confirm a 3-column layout, then resize/preview at a narrow (mobile) width to confirm the grid reflows to fewer columns.

## Source Code

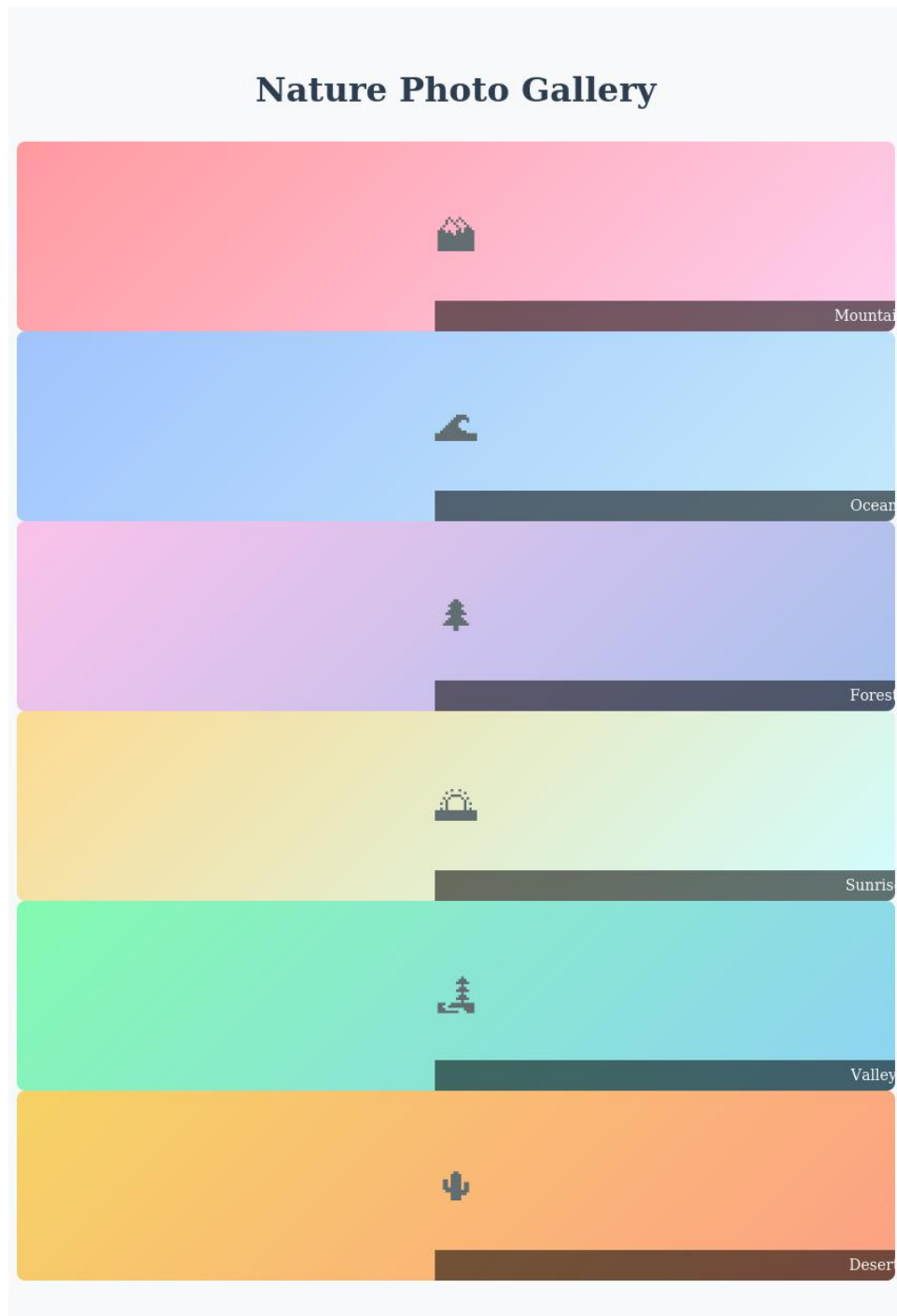
```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Lab 9 - Responsive Image Grid (Bootstrap 5)</title>
<link rel="stylesheet" href="vendor/bootstrap.min.css">
<style>
  body { background: #f8f9fa; padding: 30px 0; }
  .gallery-title { text-align: center; margin-bottom: 30px; color: #2c3e50; }
  .grid-item {
    background: #dfe6e9;
    height: 180px;
    border-radius: 8px;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 40px;
    color: #636e72;
    overflow: hidden;
    position: relative;
  }
  .grid-item .caption {
    position: absolute;
    bottom: 0;
    width: 100%;
    background: rgba(0,0,0,0.55);
    color: white;
    text-align: center;
    padding: 6px 0;
    font-size: 14px;
  }
  .g1 { background: linear-gradient(135deg,#ff9a9e,#fecfef); }
  .g2 { background: linear-gradient(135deg,#a1c4fd,#c2e9fb); }
  .g3 { background: linear-gradient(135deg,#fbc02d,#a6c1ee); }
```

```

.g4 { background: linear-gradient(135deg,#fddb92,#d1fdff); }
.g5 { background: linear-gradient(135deg,#84fab0,#8fd3f4); }
.g6 { background: linear-gradient(135deg,#f6d365,#fda085); }
</style>
</head>
<body>
<div class="container">
  <h1 class="gallery-title">Nature Photo Gallery</h1>
  <div class="row g-3">
    <div class="col-6 col-md-4">
      <div class="grid-item g1">🏔️<div class="caption">Mountains</div></div>
    </div>
    <div class="col-6 col-md-4">
      <div class="grid-item g2">🌊<div class="caption">Ocean</div></div>
    </div>
    <div class="col-6 col-md-4">
      <div class="grid-item g3">🌲<div class="caption">Forest</div></div>
    </div>
    <div class="col-6 col-md-4">
      <div class="grid-item g4">🌅<div class="caption">Sunrise</div></div>
    </div>
    <div class="col-6 col-md-4">
      <div class="grid-item g5">🏞️<div class="caption">Valley</div></div>
    </div>
    <div class="col-6 col-md-4">
      <div class="grid-item g6">🌵<div class="caption">Desert</div></div>
    </div>
  </div>
</div>
</body>
</html>

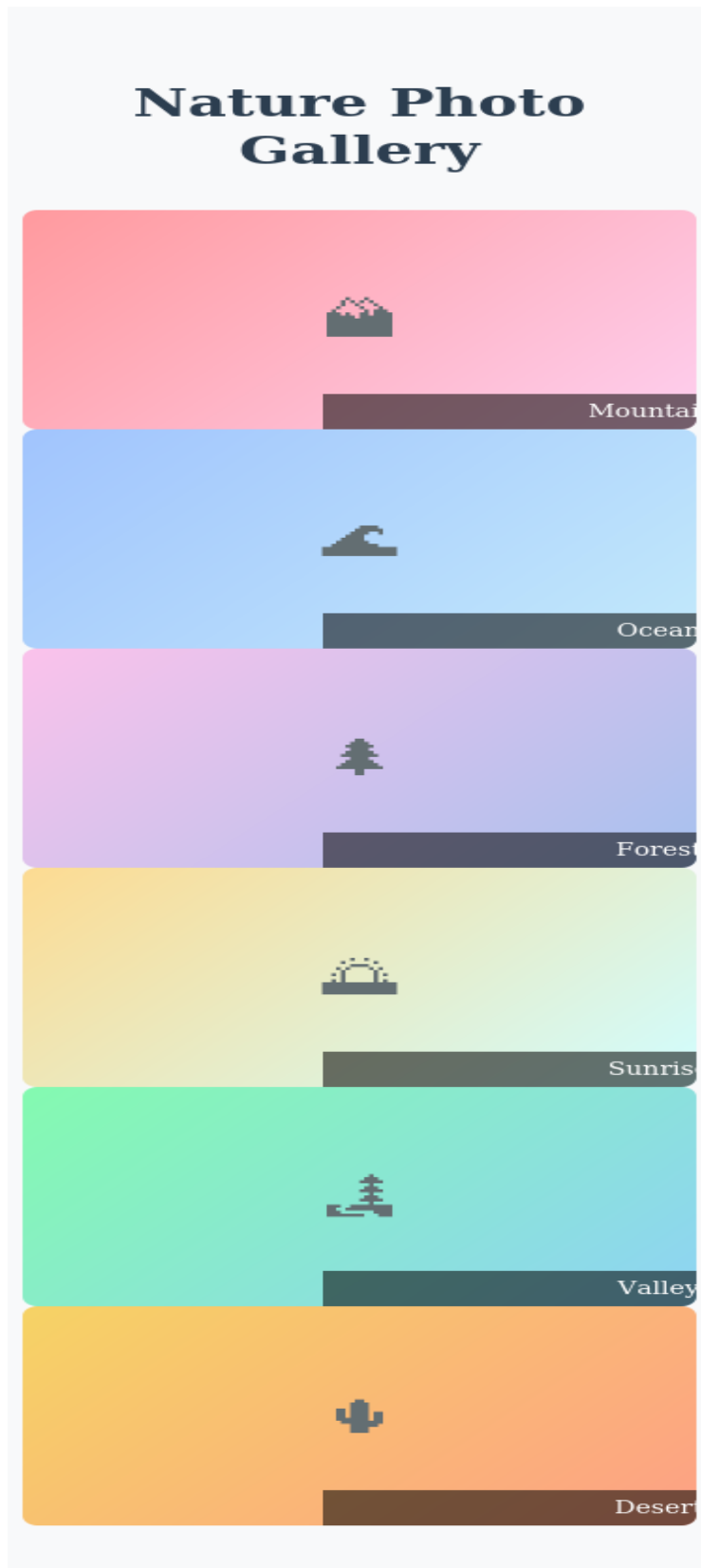
```

## Output



*Fig. 9(a): Image grid on a desktop-width viewport (3 columns)*





*Fig. 9(b): Same grid on a mobile-width viewport (2 columns) — demonstrating responsiveness*

# Experiment 10: AJAX-based Dynamic Content Loading with jQuery Fading Effects

## Aim

To write a JavaScript/jQuery program that dynamically loads content asynchronously (simulating an AJAX request) and applies jQuery visual effects such as `fadeIn()` and `fadeOut()` while displaying the content.

## Theory / Description

AJAX (Asynchronous JavaScript and XML) allows a web page to request and load data from a server in the background, without reloading the entire page. In a real deployment this is done using the Fetch API, XMLHttpRequest, or jQuery's `$.ajax()/$.getJSON()` methods.

jQuery is a JavaScript library that simplifies DOM manipulation and animation. Its effect methods — `fadeIn()`, `fadeOut()`, and `hide()/show()` — animate the opacity (and visibility) of elements over a specified duration, producing smooth transitions.

In this lab, the AJAX request/response cycle is simulated using JavaScript's `setTimeout()` to represent network latency, since it demonstrates the same asynchronous callback pattern used by real AJAX calls. The returned data is then rendered into the page and revealed using jQuery's `fadeIn()` effect.

## Procedure (Step-by-Step)

28. Include the jQuery library (`jquery.min.js`) in the HTML document.
29. Define a JavaScript array of sample article objects to act as simulated server-side data.
30. Write a `fetchDataFromServer(callback)` function that uses `setTimeout()` to invoke the callback after a short delay, mimicking an asynchronous network request.
31. Write a `renderArticles(data)` function that builds HTML markup for each article and inserts it into a content container using jQuery's `.html()` method.
32. Add a 'Load Content' button that, on click, fades out the content box, fetches the (simulated) data, re-renders it, and fades the box back in using `.fadeOut()` and `.fadeIn()`.
33. Add separate 'Fade Out' and 'Fade In' buttons that directly call `$("#contentBox").fadeOut()` and `.fadeIn()` to demonstrate the jQuery effects independently.
34. Trigger the data load once automatically when the page is ready (`$(document).ready()`) so the initial content appears with a fade-in animation.
35. Test the page by clicking each button and observing the fade transitions and the dynamically loaded article content.

## Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Lab 10 - AJAX Content Loading with jQuery Effects</title>
<script src="vendor/jquery.min.js"></script>
<style>
  body {
    font-family: Arial, sans-serif;
    background: #eef2f5;
    padding: 30px;
    margin: 0;
    display: flex;
    justify-content: center;
  }
  .container {
    background: white;
    width: 550px;
    padding: 25px 30px;
    border-radius: 8px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
  }
  h2 { color: #2c3e50; text-align: center; margin-top: 0; }
  .btn-group { text-align: center; margin-bottom: 15px; }
```

```

button {
  background: #2980b9;
  color: white;
  border: none;
  padding: 9px 18px;
  border-radius: 5px;
  cursor: pointer;
  margin: 0 5px;
  font-size: 14px;
}
button:hover { background: #21618c; }
#contentBox {
  border: 1px solid #ddd;
  border-radius: 6px;
  padding: 15px 18px;
  min-height: 140px;
  background: #f9fbfd;
}
#contentBox .article {
  margin-bottom: 12px;
  padding-bottom: 10px;
  border-bottom: 1px solid #e6eae6;
}
#contentBox .article h4 { margin: 0 0 4px; color: #2980b9; }
#contentBox .article p { margin: 0; font-size: 14px; color: #555; }
#loader {
  text-align: center;
  color: #888;
  font-style: italic;
}
</style>
</head>
<body>
  <div class="container">
    <h2>Dynamic News Loader (AJAX + jQuery)</h2>
    <div class="btn-group">
      <button id="loadBtn">Load Content (AJAX)</button>
      <button id="fadeOutBtn">Fade Out</button>
      <button id="fadeInBtn">Fade In</button>
    </div>
    <div id="contentBox">
      <p id="loader">Click "Load Content" to fetch articles...</p>
    </div>
  </div>

<script>
  // Simulated server-side data source (in a real AJAX app this would come from an API endpoint)
  const serverData = [
    { title: "New Campus Library Opens", body: "The university inaugurated its new digital library with over 50,000 e-resources." },
    { title: "Tech Fest 2026 Announced", body: "Annual technical festival will be held in September featuring hackathons and workshops." },
    { title: "Placement Drive Results", body: "Over 85% of the graduating batch received placement offers this year." }
  ];

  // Simulates an AJAX call using a delayed callback (represents a network request)
  function fetchDataFromServer(callback) {
    $("#loader").text("Loading content from server...");
    setTimeout(function () {
      callback(serverData);
    }, 800); // simulated network latency
  }

  function renderArticles(data) {
    let html = "";
    data.forEach(item => {
      html += `<div class="article"><h4>${item.title}</h4><p>${item.body}</p></div>`;
    });
    $("#contentBox").html(html);
  }

```

```

$(document).ready(function () {
    $("#loadBtn").click(function () {
        $("#contentBox").fadeOut(200, function () {
            fetchDataFromServer(function (data) {
                renderArticles(data);
                $("#contentBox").fadeIn(600);
            });
        });
    });

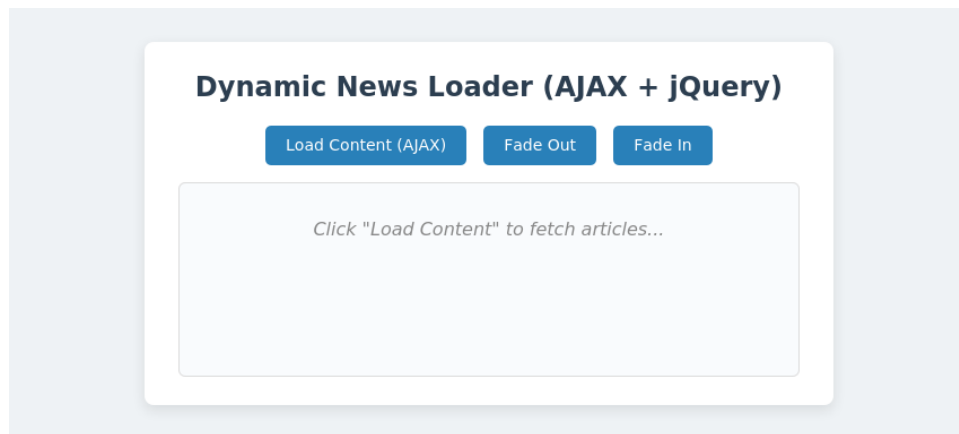
    $("#fadeOutBtn").click(function () {
        $("#contentBox").fadeOut(600);
    });

    $("#fadeInBtn").click(function () {
        $("#contentBox").fadeIn(600);
    });

    // Auto-trigger the AJAX load once on page load so the output demonstrates the result
    fetchDataFromServer(function (data) {
        renderArticles(data);
        $("#contentBox").hide().fadeIn(800);
    });
});
</script>
</body>
</html>

```

## Output



*Fig. 10: Dynamically loaded articles displayed after the simulated AJAX call and jQuery fade-in effect*